

UNIVERSITY OF LEEDS

COMP1850 Programming

Week 5 Session 1

Object Oriented Programming & Classes

Recap

We have introduced the **procedural programming** technique:

- Based around the concept of **functions**
- Each function performs a single, well-defined part of the overall task (Single Responsibility Principle)
- Functions can be developed and tested independently
- Functions are ‘black boxes’; we can use them without needing to know the details of their implementation
- Functions make it easier for us to organize and reuse code

But this is not the only approach to developing software...

Today's Aims

UNIVERSITY OF LEEDS

- To explore the **object-oriented programming** (OOP) paradigm and the fundamental concepts of OOP
- To understand what classes and objects are, and how classes can form the basis for software
- To understand how a class is defined in Python

The Procedural Paradigm

UNIVERSITY OF LEEDS

- A **task-oriented** view of the system
- We decompose a larger task into smaller sub-tasks, until those sub-tasks are small enough to be implemented as functions
- Data flows into functions via parameters, and out of functions via the return value (for example)
- Developer focus is on the *actions that those functions perform*, rather than the data they manipulate

Systems Built From Objects

UNIVERSITY OF LEEDS

Real-world systems can also be viewed as **collections of objects**

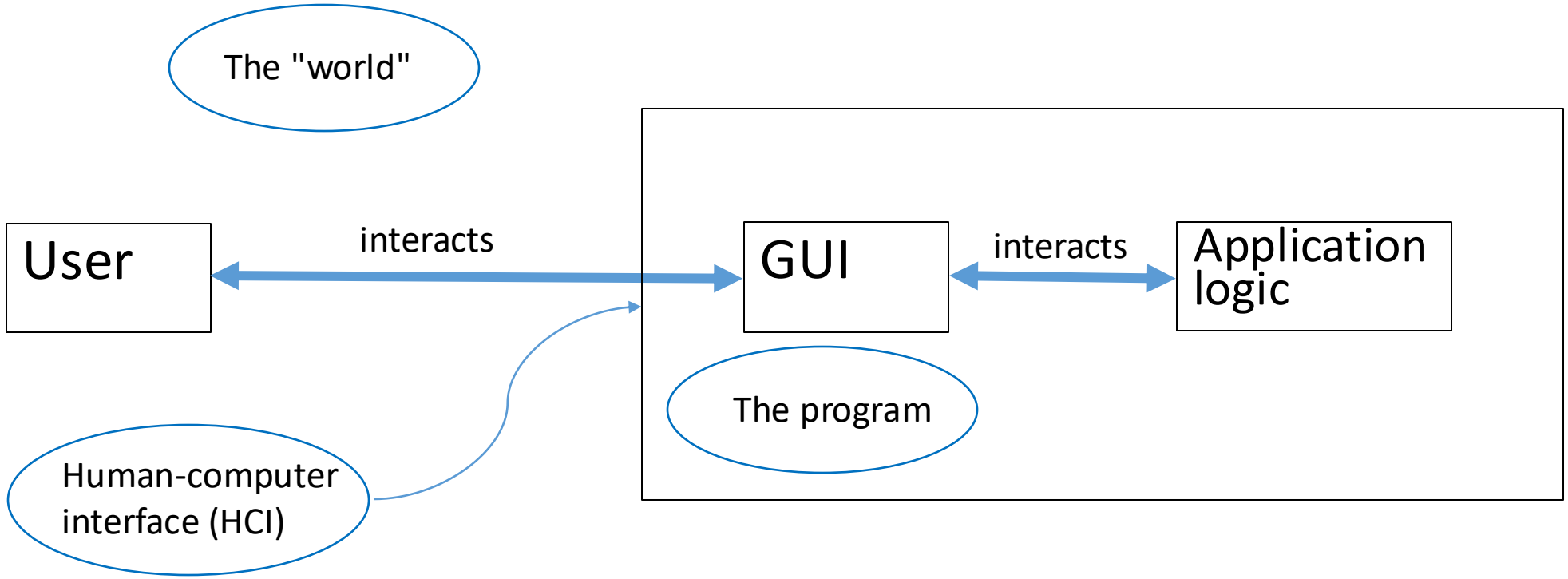
- That model components of the system as distinct items
- That interact according to specific rules

There are many cases where this view of a system is a more natural model for the problem we are solving.

It can also allow for a more natural separation of the system into its various components...

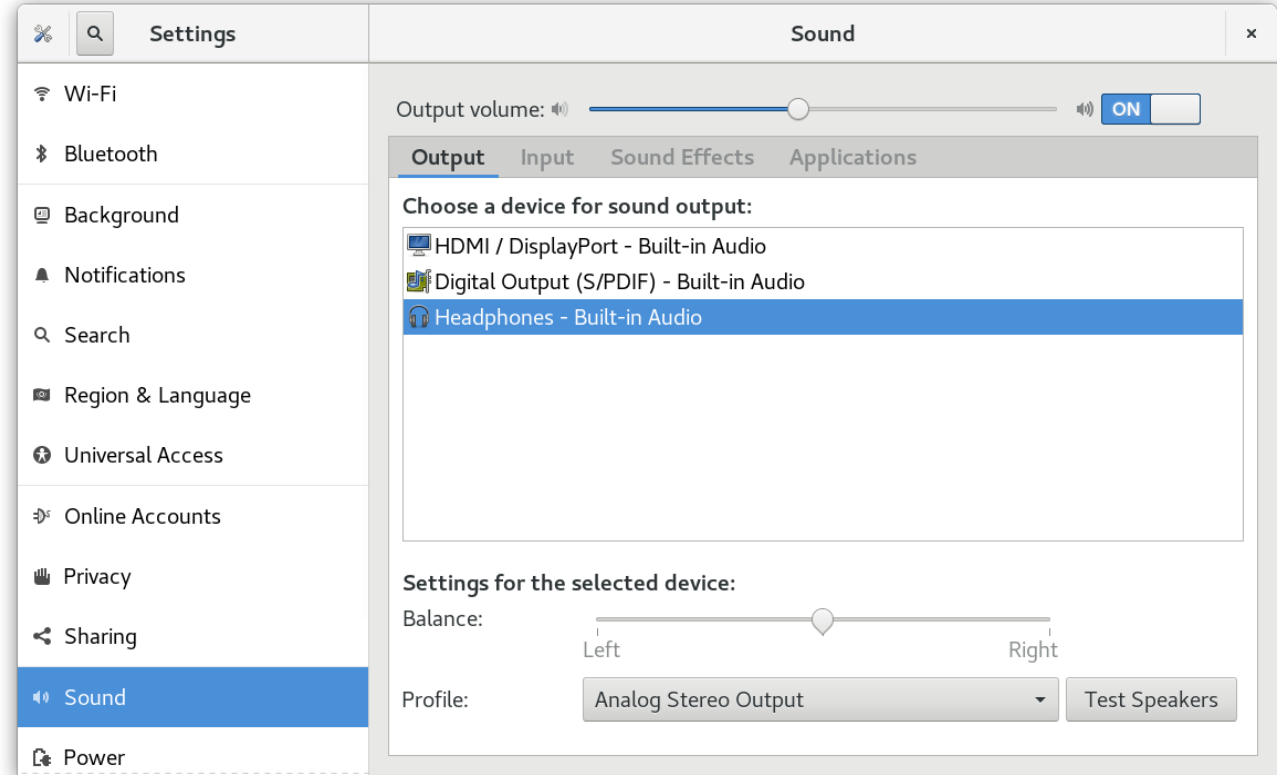
Example: Graphical User Interfaces

UNIVERSITY OF LEEDS



Example: Graphical User Interfaces

UNIVERSITY OF LEEDS



GUIs consist of **objects of different types**
(Window, Button, Switch, Options List, Slider, etc...)

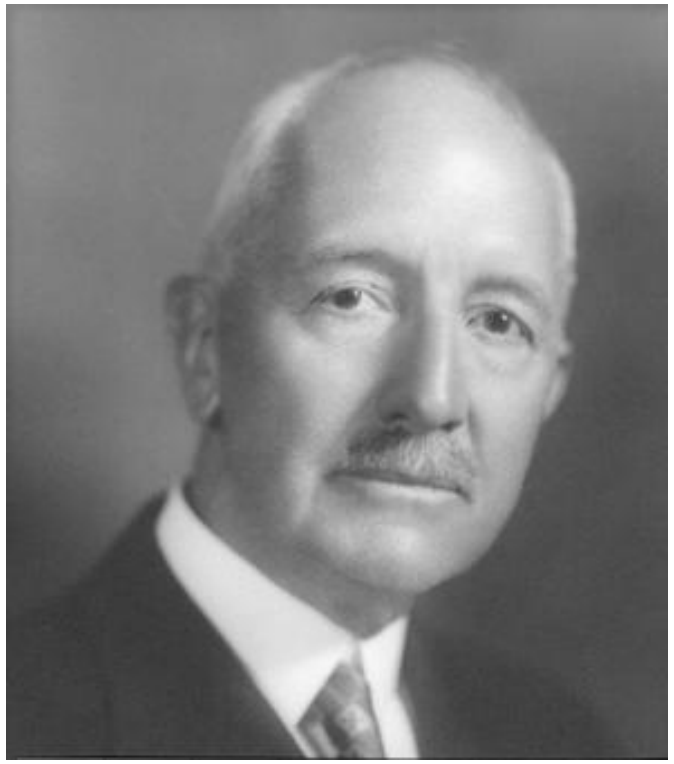
The OOP Paradigm

UNIVERSITY OF LEEDS

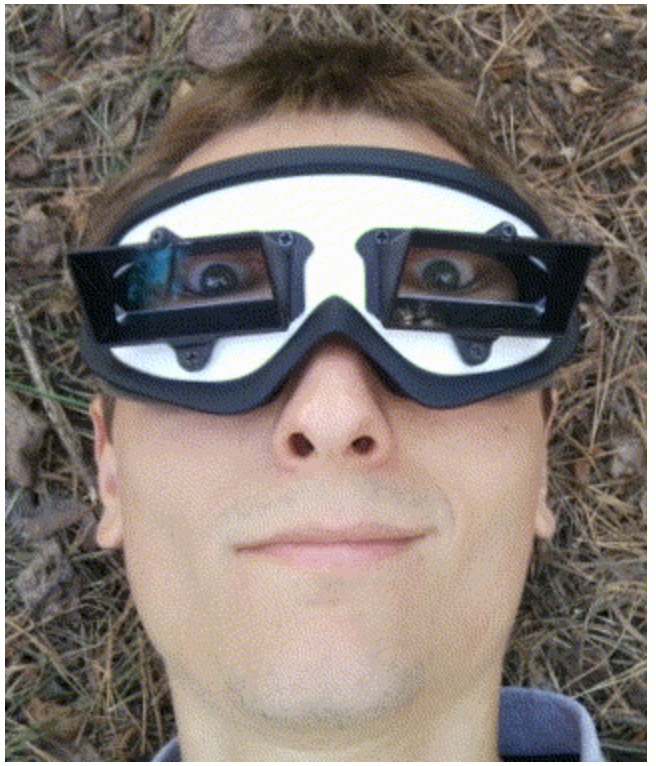
- Real-world systems can be modelled as collections of objects
  - That contain data (state) related to that object
  - That we interact with by sending ‘messages’ to an object
- A message is either
  - A request to perform an operation required by the system
  - The response to such a request
- This is fundamentally different from the procedural viewpoint, and adjusting your perspective can be challenging...

Inverting Your Perspective

UNIVERSITY OF LEEDS

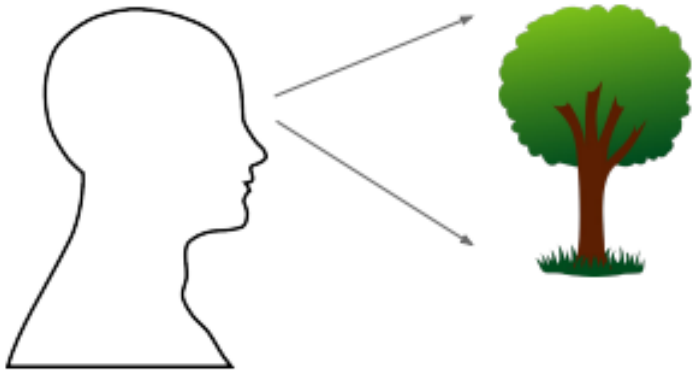


American psychologist George M. Stratton

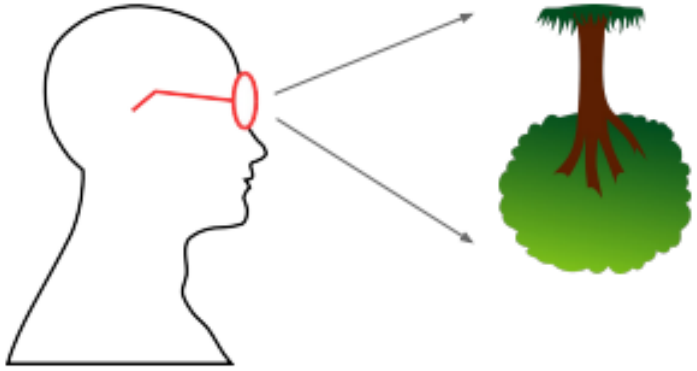


Upside-down goggles, similar to those used by Stratton in his experiments

Inverting Your Perspective



George M Stratton, 1896:
"Some Preliminary Experiments in Vision"
Psychological Review



VERBS → nouns

PROCEDURES → objects within procedures

Procedural view

NOUNS → verbs

OBJECTS → procedures within objects

OOP view

Objects & Classes

UNIVERSITY OF LEEDS

- Objects are ‘things’ in the world that have distinct identities, attributes and behaviour
- Objects that are characterised by the same attributes and behaviour can be grouped into one category, called a **class**
- A class is therefore a way of describing the similarities exhibited by a particular set of objects
- An object is a specific **instance** of a class
  - Two instances of the same class will behave in the same way
  - They can still look different because their attributes have different values

Quiz: Object or Class?

UNIVERSITY OF LEEDS

Alex Jones, studying Computer Science

Leeds United FC

Student

Lab Session

Week 1.5, Session 1 of COMP1850

Football Club

A Clock Class

UNIVERSITY OF LEEDS

- What attributes (nouns) could represent the state of a Clock?
  - hours
  - minutes
  - seconds
- What behaviour (verbs) could a Clock have?
  - Display the time
  - Set the time
  - Tick (i.e., update the time)



Task 1: A Student Class

UNIVERSITY OF LEEDS

`week1.5/session1/task1/`

Edit `Student.md` and modify it so that it lists

- Attributes that could be used to represent the state of a Student
- Behaviours that would be associated with a Student class, given your choice of attributes

Software Classes

UNIVERSITY OF LEEDS

- A class in software is an **abstraction** of a set of objects found in the problem domain; it provides a **simplified representation** of their attributes and behaviour
- A software class represents these attributes using a set of **instance variables** or **fields**
- A software class represents the behaviour with **methods**, which are defined in the same way as functions

Software Objects

UNIVERSITY OF LEEDS

- A software class is a ‘template’ for making objects
- At run time, a program will **instantiate** the class (create an object)
  - This involves giving values to the object’s fields
- After an object has been created, the program interacts with it
  - By accessing its fields
  - And by calling its methods

Abstraction & Encapsulation

Two important general principles of OOP

Abstraction

- We define a class using the minimum necessary data (state)
- We define the minimum set of methods as an interface to the class

Encapsulation

- We store data that represents object state *inside* the class
- We define methods that operate on that state *inside* the class

Data Hiding

UNIVERSITY OF LEEDS

- A third principle of OOP
- Goal is to have a distinction between **private implementation details** and the **public interface** that is available to class users
- Typically, we make the fields private and provide controlled access to them via methods in the public interface
- Can’t really enforce this in Python, but other OO languages provide strong support for this concept
  - More on this next year!

Python and OOP

- Objects are central to Python; *everything* you manipulate in a Python program is represented as an object
- Python fully supports, but does not require, a similar object-oriented approach in the code that you write yourself
- You can structure your code using only classes, or program procedurally (as we've been doing so far), or mix both paradigms

Everything is an Object

UNIVERSITY OF LEEDS

Every item in Python is an object, with:

- An associated class definition
- An internal data representation
- A set of methods to handle interactions with the object

We can use `type()` to check the type of object

We can use `dir()` to check the methods that apply to that object

Defining a Python Class

UNIVERSITY OF LEEDS

- Start with the `class` keyword
- This is followed by the class name, then a colon
  - Names should be in **upper camel case**
  - Examples: `MainWindow`, `StaffMember`, `LibraryBook`
- Beneath this, on indented lines, we write the methods
  - Standard Python style is to use **snake case** for method names
  - Examples: `load_data`, `set_time`

Python style guide: https://peps.python.org/pep-0008/

Example

UNIVERSITY OF LEEDS

- Let's create a simple class to represent a point in two-dimensional space (e.g., for use in a graphics application):

```
class Point:
    """
    A point in 2D space.
    """
```

- This has a docstring (important!), but no fields or methods (yet)
- Nevertheless, it is a valid Python class!

Next Step: The Constructor

UNIVERSITY OF LEEDS

- The **constructor** handles object creation
- It creates the fields used by an object, and can initialize those fields with values supplied by the user of the class
- The constructor *must* have the name `__init__()`

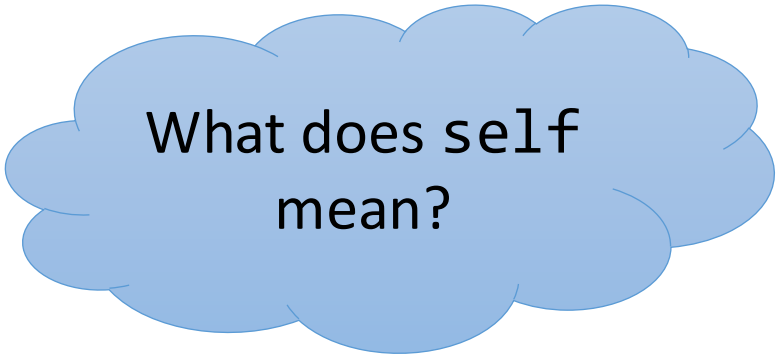
Note the use of **double underscores** before *and* after `init`

Point Constructor

UNIVERSITY OF LEEDS

```
class Point:
    """
    A point in 2D space.
    """

    def __init__(self, x, y):
        """
        Creates a Point, given x & y coordinates.
        """
        self.x = x
        self.y = y
```



Meaning of self

- The special `self` parameter refers to 'the object itself' – i.e., the object we are creating
  - You can name it something different, but nobody actually does this!
- `self` is **always the first parameter** in the parameter list
- The remaining parameters represent the inputs required in order to create an object
- Inside the constructor, **'`self.`' must be used as a prefix** when referring to the fields we are creating

Task 2

UNIVERSITY OF LEEDS

`week1.5/session1/task2/`

- Investigate the problems with two bad class definitions
- See `README.md` for full details of what to do

Things to Note

UNIVERSITY OF LEEDS

- There is nothing in the class to restrict the values or types of values used as constructor arguments when creating a `Point`
- There is nothing to stop us introducing additional fields after we have created a `Point` object
- `Point` doesn't have a private implementation that is separate from its public interface; everything is accessible to users

We will address some of these issues next session…

Adding Other Methods

UNIVERSITY OF LEEDS

```
import math

class Point:
    ...
    def distance(self):
        """
        Returns distance from this Point to origin.
        """
        return math.hypot(self.x, self.y)
```

(See https://docs.python.org/3/library/math.html#math.hypot)

Things to Note

UNIVERSITY OF LEEDS

- All methods need a `self` parameter, not just the constructor
- `self` refers to the object on which the method is called
- All methods must use the '`self.`' prefix when referring to the fields that were created by the constructor
- Methods should be given descriptive docstrings

Using The Class

UNIVERSITY OF LEEDS

constructor is invoked using **name of class**, not `__init__`

constructor `self` parameter refers to the created object, referenced here as p

```
p = Point(4.5, 7.2)

print(p.x, p.y)
print(p.distance())
```

`self` parameter of `distance()` refers to object on the left of the dot operator…

…so nothing is passed in a as an argument when calling the method

Task 3

UNIVERSITY OF LEEDS

`week1.5/session1/task3/`

- Implement the `Point` class in `point.py`
- Add a `distance_to()` method that computes and returns the distance from the `Point` object receiving the method call, to another `Point` object specified as a method parameter
- Add code to `point.py` that demonstrates use of the class and its `distance_to()` method

Importing Class Definitions

UNIVERSITY OF LEEDS

- It is customary to put class (and function) definitions in separate **modules** (i.e., different files to the ones where the class or function is actually used)
- This facilitates reuse of classes and functions
  - Across the different modules that make up a single application
  - Across multiple applications
- Definitions are **imported** from a module into any other module that needs to use them, by means of `import` statements

Import Examples

UNIVERSITY OF LEEDS

geometry.py

```
class Point:
    """
    A point in 2D space.
    """
    def __init__(self, x, y):
        self.x = x
        self.y = y
    ...
```

example1.py

```
import geometry

p = geometry.Point(1.5, 4.2)
...
```

example2.py

```
from geometry import Point

p = Point(1.5, 4.2)
...
```

Task 4

UNIVERSITY OF LEEDS

`week1.5/session1/task4/`

- Copy your `Point` class from Task 3 into `geometry.py`
- Define a new class, `Circle`, in `geometry.py`
  - With a field to represent radius
  - With a field to represent the centre coordinates (as a `Point`)
  - With methods to compute circle area & perimeter
- Write a separate program in `circledemo.py` that creates a `Circle` object and demonstrates its functionality

(See README for full details)

Summary

UNIVERSITY OF LEEDS

We have

- Introduced object-oriented programming, which has a very different perspective from the procedural programming approach
- Discussed how we can model the world as a collection of **objects**, which belong to distinct categories or **classes**
- Noted that software classes are **abstractions** of classes in the real world, and that they **encapsulate** state & behaviour as a set of **fields** and **methods**
- Learned the syntax for defining and using Python classes

Learning More

UNIVERSITY OF LEEDS

How to Think Like a Computer Scientist, Chapter 17: Classes and Objects: The Basics

(Specifically, subsections 17.1 – 17.7)